Custom fabricator — project-based pipeline rebuild in HubSpot

A custom fabricator ran a bid-driven business through a pipeline inherited from its old tool — sales and production stages tangled together. The rebuild separated winning the work from doing the work, and used the objects HubSpot already had.

Paul Maxwell, CEO, RevOps HQ July 2026

ABSTRACT

A custom fabrication studio — bid-driven, long cycles, every job a one-off — migrated to HubSpot from a legacy project tool and carried its pipeline across one-to-one: dozens of stages that mixed *sales* progression with *production* approvals, so forecasting meant nothing. A project business has two lifecycles: winning the work and doing the work. This is a first-principles account of separating them in HubSpot without minting a Project custom object — a streamlined deal pipeline for sales, the native Ticket object repurposed as the project record, a separate change-order pipeline, and stage requirements as hard gates.

1 A PROJECT BUSINESS HAS TWO LIFECYCLES

The mistake a project business makes in any CRM is treating the pipeline as one thing. There are two: the *sales* lifecycle that ends when a bid is won, and the *production* lifecycle that begins there — fabrication, change orders, delivery. Collapse them into one pipeline and every report lies, because "stage" means both "how likely is this to close" and "where is it on the shop floor."

That is exactly what a one-to-one migration produced here: the old tool's pipeline, carried across intact, with sales stages and production-approval stages and change-order stages and half a dozen lost/cancel variants all in one funnel. Forecasting was impossible, because the pipeline wasn't a forecast — it was a work log wearing a forecast's clothes.

THE FIRST PRINCIPLE

Separate the sales pipeline from the production lifecycle. The deal ends at won; the work starts at won. Two records, two lifecycles — and the object for the second one is a decision, not a reflex.

2 THE DEAL ISN'T THE WORK

One sold project spawns production tracking, work orders, and change orders. The reflex is a **Project custom object** — but custom objects spend a limited Enterprise budget and get weaker native pipeline and reporting support than the standard objects. Before minting one, ask what standard object already behaves like the thing you need. Here two did.

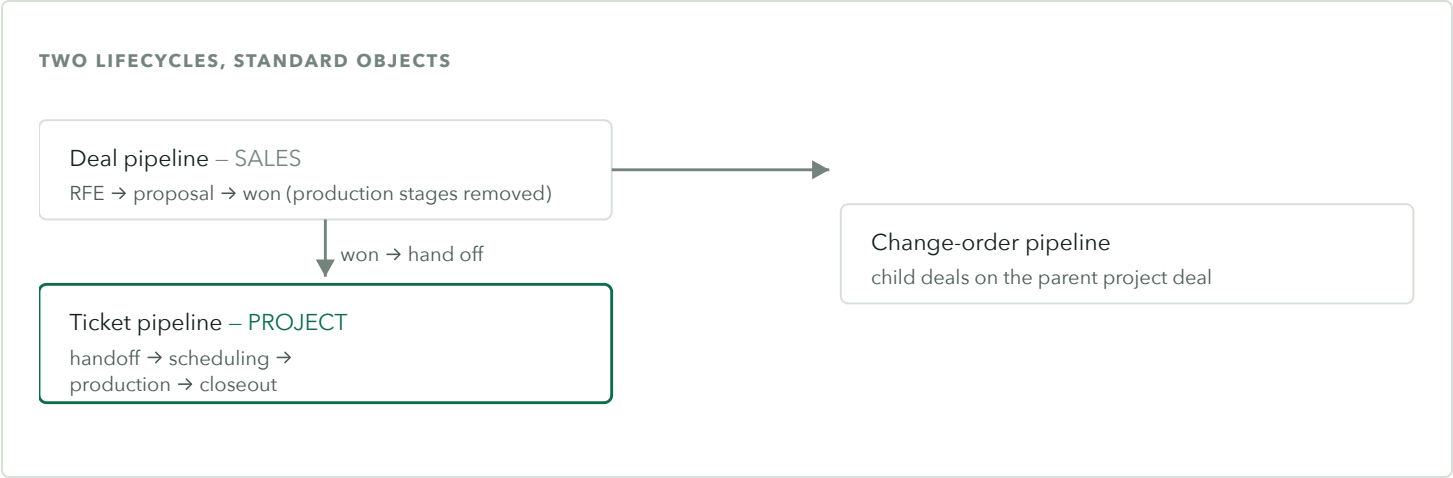

Figure 1. The sold deal (sales) hands off at won to a Ticket pipeline used as the project record (production), with a separate change-order pipeline of child deals – all standard objects, no Project custom object.

3 THE TICKET IS THE PROJECT

The native **Ticket** object — usually thought of as support — is a pipeline with stages, ownership, and permissions. That is exactly a project record. Repurposed, it runs the production lifecycle (handoff → scheduling → production → closeout) with full native reporting and no custom-object cost. A single discriminator property on the Company (customer / vendor / partner) replaced several other proposed custom objects; only one genuinely spatial concept — a delivery Location, many-to-many to deals and tickets — earned an object of its own.

> **THE REFLEX TO RESIST**
>
> Not everything that is a "thing" needs a custom object. A discriminator property, a repurposed standard object, or a parent/child association often models it with better native support and no budget cost. Custom objects are a last resort, not a first move.

4 CHANGE ORDERS, AND A REAL HUBSPOT LIMIT

A change order is a mid-project scope change, so it's modeled as a separate pipeline of **child deals** associated to the parent project deal. Adding scope is straightforward. *Deducting* scope hits a genuine platform limit: HubSpot rejects negative line items, so a reduction can't be a quote revision. The workaround is honest and explicit — adjust a contracted-value property, issue the credit in the accounting system, and let a one-way sync surface the net — rather than faking it with a "void and duplicate" that wrecks the client's paper trail.

5 STAGE REQUIREMENTS ARE THE GATES

A bid can't reach a client until it clears internal gates — production buy-in, a margin check, a shop-capacity check. In HubSpot those are **stage requirements**: a deal can't advance past "capacity approved" until production confirms capacity; a ticket can't enter production until every work order is staged. Required properties — including required *file* uploads — enforce that the document exists before the stage moves. The gate is enforced by the system, not remembered by a person.

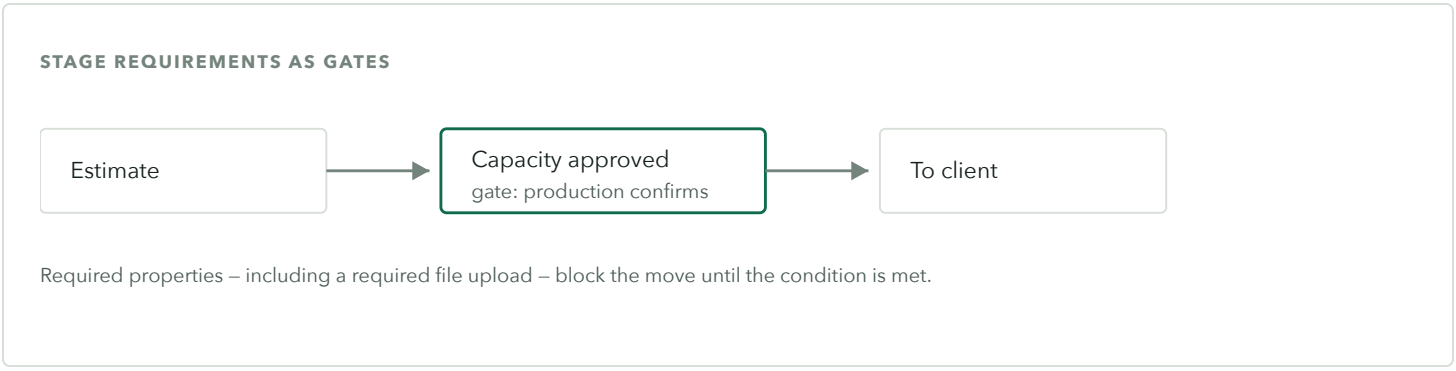

Figure 2. Stage requirements turn a pipeline into a controlled process: the deal cannot advance until the gate's conditions (a confirmation, a required file) are satisfied.

6 WHAT CHANGED

The sales pipeline forecasts again, because it only holds sales stages. Production runs on its own Ticket lifecycle with real reporting. Change orders are tracked as child deals against the project, with an honest path for deductions. And the gates that used to live in people's heads — capacity, margin, documentation — are enforced by the system. The firm got a platform that matches how projects actually move, built almost entirely from standard objects.

Details are generalized and figures are representative of the engagement; specifics vary by shop.

7 THE MODEL, GENERALIZED

Any project- or job-based business faces the same exercise:

- **Split sales from delivery.** The deal ends at won; the work is a separate lifecycle.
- **Repurpose a standard object before minting a custom one.** A Ticket pipeline is a project record; a discriminator property replaces parallel objects.
- **Model change orders as child deals** — and handle the negative-line-item limit honestly.
- **Use stage requirements as gates** — required fields and files — so process is enforced, not remembered.

RELATED STUDIES

The same "model before software" discipline runs through migrating an *advisory firm off a legacy CRM*, resolving *identity* at an integration boundary, and representing a *dealer channel* where the buyer isn't the user.